

Guía de estudio - React Router II



¡Hola! Te damos la bienvenida a esta nueva guía de estudio.

¿En qué consiste esta guía?

La siguiente guía de estudio tiene como objetivo recordar y repasar los contenidos que hemos visto en clase, dentro de los que se encuentran:

- Crear aplicaciones web en React que tengan sistemas de navegación, redirección y captura de parámetros en url.

¡Vamos con todo!



Tabla de contenidos

La clase active	3
El componente NavLink	3
¡Manos a la obra! - La clase active	6
El hook useNavigate()	6
¡Manos a la obra! - Redireccionamiento programático	8
El hook useParams()	8
¡Manos a la obra! - Obteniendo parámetros de la URL	10
Rutas protegidas	10
¡Manos a la obra! - Identifíquese	16
Preguntas de cierre	16



¡Comencemos!

La clase active

Existen consideraciones visuales que aportan a una mejor experiencia de usuario, entre ellas, personalizar el estilo de la opción que representa la ruta actual.

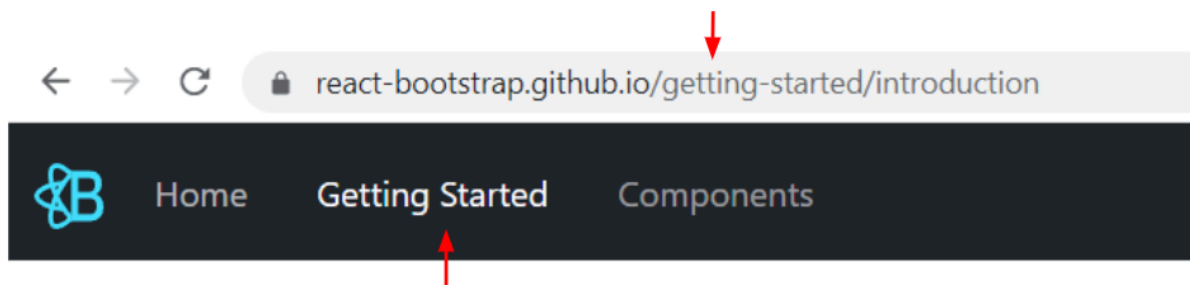


Imagen 1. Ejemplo de coincidencia entre ruta y opción
Fuente: Desafío Latam

Este pequeño, pero importante cambio visual, ayudará a nuestros usuarios a estar mejor ubicados mientras navegan en nuestra aplicación.

El componente NavLink

React Router nos provee un componente que nos ayudará a saber cuál de las opciones de navegación corresponde a la ruta consultada.

```
<NavLink  
  className={({ isActive }) => (isActive ? "nombre-de-la-clase" : undefined) }  
  to="/" >  
    Home  
</NavLink>
```

A diferencia del ya conocido componente Link, este componente tiene integrado un atributo **isActive** que nos devolverá **true** o **false** si la ruta está siendo consultada.

Para acceder a este atributo, necesitamos definir una función como valor del atributo **className**, la cual tendrá como parámetro un objeto con la propiedad **isActive** y deberá retornar el nombre de la clase que será asignada a este elemento.



Más información en: <https://reactrouter.com/en/main/components/nav-link>

Para ver en la práctica el uso de este componente, crea una nueva aplicación de React e instala React Router. Luego, realicemos los siguientes pasos:

1. Agrega una clase `.active` en ***index.css***

```
.App {  
  font-family: sans-serif;  
  text-align: center;  
}  
  
.active {  
  color: darkgreen;  
}
```

2. Crea un componente **Navbar** con 2 opciones: Home y Personajes

```
import { NavLink } from "react-router-dom";  
  
export default function Navbar() {  
  const setActiveClass = ({ isActive }) => (isActive ? "active" : undefined);  
  return (  
    <div>  
      <NavLink className={setActiveClass} to="/">  
        Home  
      </NavLink>  
      {" - "}  
      <NavLink className={setActiveClass} to="/personajes">  
        Personajes  
      </NavLink>  
    </div>  
  );  
}
```

3. Modifica el código de **App.jsx** para agregar el enrutador e incluir el Navbar creado

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Navbar from "../components/Navbar";

export default function App() {
  return (
    <div className="App">
      <BrowserRouter>
        <Navbar />
      </BrowserRouter>
    </div>
  );
}
```

El resultado de lo anterior lo podremos apreciar levantando la aplicación y visitando las rutas

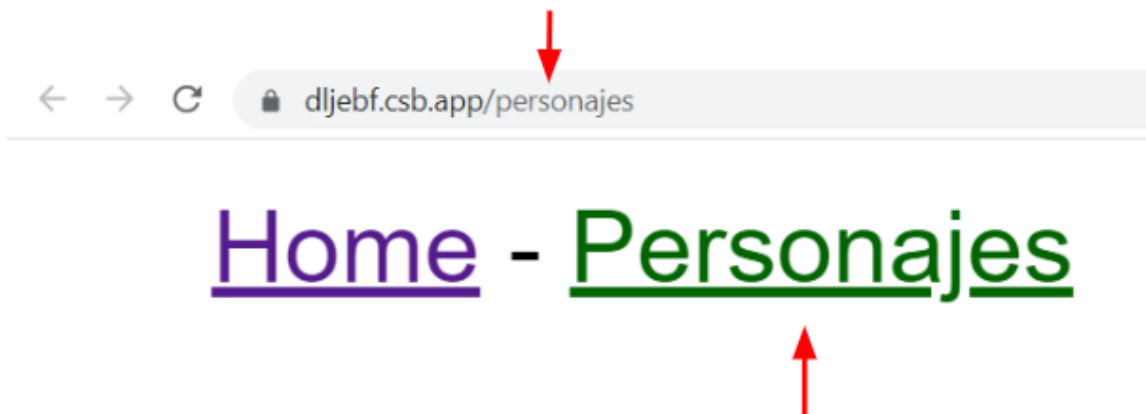


Imagen 2. Ejemplo de la clase active
Fuente: Desafío Latam



¡Manos a la obra! - La clase active

Continuando con nuestra aplicación:

1. Agrega una nueva opción en el Navbar.
2. Crea una nueva clase para definir los estilos de la ruta activa.
3. Crea una nueva clase para definir los estilos de las rutas que no estén activas.
4. Comprueba que las nuevas clases son asignadas correctamente al consultar las rutas.

El hook `useNavigate()`

En algunos casos podríamos necesitar redireccionar a nuestros usuarios sometiendo esta acción a una condición o simplemente agregar en la URL de destino alguna variable.

A esto se le conoce como un redireccionamiento programático y en React Router podemos realizarlo utilizando el hook `useNavigate()`.



Más información en: <https://reactrouter.com/en/main/hooks/use-navigate>

Veamos un ejemplo del redireccionamiento programático creando una vista que reciba del usuario un id numérico y redirija a una ruta `/personajes/:id`

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

export default function Home() {
  const [id, setId] = useState("");
  const navigate = useNavigate();
  const irAPersonajes = () => {
    navigate(`/personajes/${id}`);
  };
  return (
    <div className="mt-5">
      <h1>React Router II</h1>
      <input
        type="number"
        value={id}
        onChange={({ target }) => setId(target.value)}
      />
    </div>
  );
}
```

```
    />
    <button onClick={ irAPersonajes }>buscar</button>
  </div>
);
}
```

Ahora crearemos una vista Personajes que simplemente retorne un 

```
export default function Personajes() {
  return (
    <div className="mt-5">
      <h1>
```

Finalmente, hay que agregar en el componente App las rutas relacionadas a las vistas creadas

```
import './styles.css';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';

import Home from './views/Home';
import Personajes from './views/Personajes';

export default function App() {
  return (
    <div className="App">
      <BrowserRouter>
        <Navbar />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/personajes/:id" element={<Personajes />} />
        </Routes>
      </BrowserRouter>
    </div>
  );
}
```

Probemos nuestra aplicación escribiendo un id en el input y presionando el botón OK:

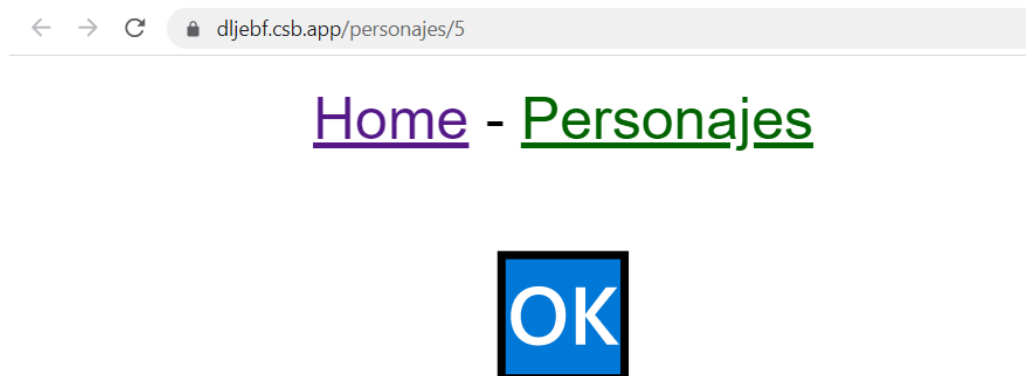


Imagen 3. Redireccionamiento programático
Fuente: Desafío Latam



¡Manos a la obra! - Redireccionamiento programático

Continuando con nuestra aplicación:

1. Cambia el input por un selector del 1 al 10.
2. Condiciona el redireccionamiento en función de que la variable **id** contenga algún valor.
3. Comprueba los cambios en el navegador.

El hook `useParams()`

Con React Router podemos definir en el **path** de una ruta el reconocimiento de parámetros:

```
<Route path="/categoria/:nombre" element={<Componente />} />
```

Por ejemplo, si se consulta: `/categoria/pib`

El parámetro **nombre** tomará como valor: `"pib"`

Para que un componente acceda a los parámetros de la URL, se necesita ocupar el hook `useParams()`



Más información aquí: <https://reactrouter.com/en/main/hooks/use-params>

Al ser ejecutado, este hook nos devuelve un objeto con todos los parámetros.

Veamos un ejemplo capturando el **id** del personaje que logramos incluir en la URL a través de un redireccionamiento programático

Modifica el código de la vista *Personajes* para que reciba y muestre el parámetro en lugar del 

```
import { useParams } from "react-router-dom";

export default function Personajes() {

  const { id } = useParams();

  return (
    <div className="mt-5">
      <h1>{ id }</h1>
    </div>
  );
}
```

Si probamos nuevamente el redireccionamiento programático, ahora veremos que se visualiza el valor del id en el componente

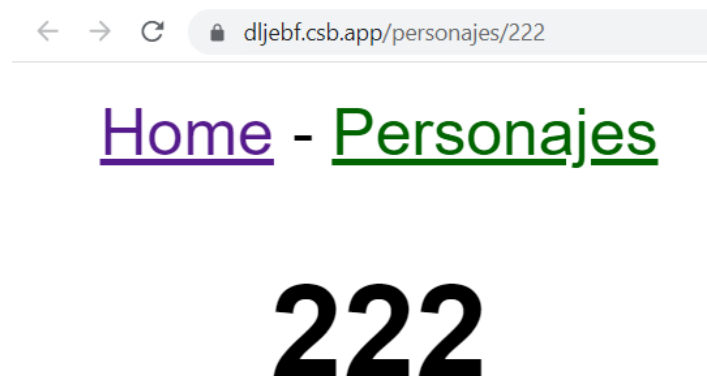


Imagen 4. El id recibido como parámetro en el componente
Fuente: Desafío Latam



¡Manos a la obra! - Obteniendo parámetros de la URL

En la vista Personajes:

1. Utiliza el **id** recibido en parámetros para consultar la [API de Rick And Morty](#) y obtener la información del personaje que coincida con el id.
2. Agrega React Bootstrap al proyecto para ocupar el componente Card y mostrar la información del personaje.
3. Condiciona el componente Card a solo mostrarse en caso de que haya obtenido un personaje con el id desde la vista Home, de lo contrario, muestra en el componente un texto indicando que no se encontró ningún personaje con el id ingresado.

Rutas protegidas

Cuando desarrollamos sistemas que incluyan cuentas de usuario, normalmente tendremos secciones restringidas a solo aquellas que tengan una sesión de usuario activa.

Para lograr esto, podemos condicionar la ruta protegida para que valide la existencia previa de una sesión antes de renderizar su componente en cuestión.

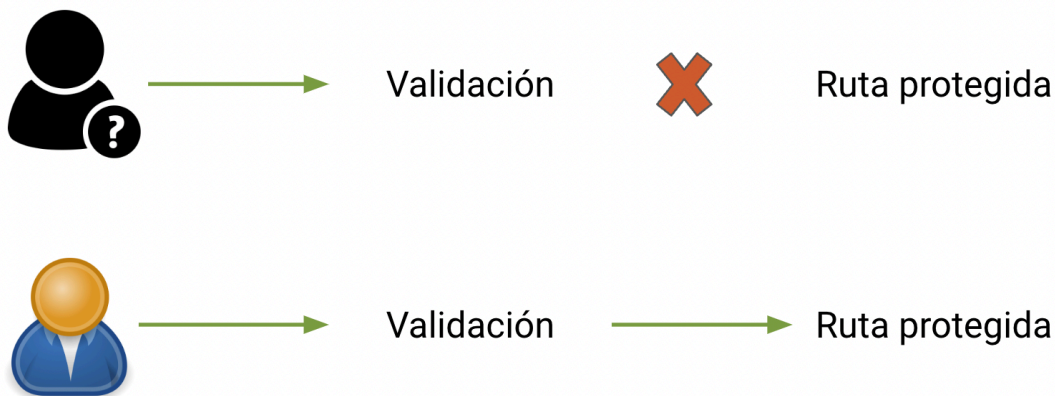


Imagen 5. Diagrama de rutas protegidas
Fuente: Desafío Latam

Llevemos este caso a la práctica con una pequeña aplicación que contenga 3 rutas:

- Home
- Administración

- Login

Siendo la **Administración** la ruta protegida o privada, mientras que **Home** y **Login** serán rutas que denominaremos como públicas.

Crea un nuevo proyecto de React e instala React Router DOM para realizar el siguiente ejercicio.

Iniciemos creando nuestro Navbar. Para eso, utiliza lo siguiente:

Componente Navbar:

```
const Navbar = () => {

  const setActiveClass = ({ isActive }) => (isActive ? "active" : undefined);

  return (
    <div>
      <nav>

        <NavLink className={setActiveClass} to="/"> Home </NavLink>

        <NavLink className={setActiveClass} to="/admin"> Administracion </NavLink>

        <NavLink className={setActiveClass} to="/login"> Login </NavLink>

      </nav>
    </div>
  );
};
```

Estilos para el componente Navbar

```
nav {
  display: flex;
  justify-content: center;
  gap: 10px;
  padding: 30px;
}

.active {
  color: darkgreen;
  font-weight: bold;
}
```

Ahora creemos nuestros 3 componentes relacionados con las 3 rutas de nuestro navbar.

src\views\Administracion.jsx

```
const Administracion = () => {  
  return <div>Administración</div>;  
};  
export default Administracion;
```

src\views\Home.jsx

```
const Home = () => {  
  return <div>Home</div>;  
};  
export default Home;
```

src\views>Login.jsx

```
const Login = () => {  
  return <div>Login</div>;  
};  
export default Login;
```

Creamos el **userContext** con un usuario que por defecto estará en null

src\context\userContext.jsx

```
import { createContext, useState } from "react";  
  
export const UserContext = createContext();  
  
const UserProvider = ({ children }) => {  
  const [user, setUser] = useState(null);  
  
  return (  
    <UserContext.Provider value={{ user, setUser }}>  
      {children}  
    </UserContext.Provider>  
  );  
};  
export default UserProvider;
```

Configuramos nuestro **main.jsx** con nuestro **UserProvider**

src\main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";
import UserProvider from "./context/userContext.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <UserProvider>
        <App />
      </UserProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

Finalmente, configuramos nuestro App.jsx.

```
import { useContext } from "react";
import { Navigate, Route, Routes } from "react-router-dom";
import Navbar from "./components/Navbar";
import { UserContext } from "./context/userContext";

import Administracion from "./views/Administracion";
import Home from "./views/Home";
import Login from "./views/Login";

const App = () => {
  const { user } = useContext(UserContext);

  return (
    <div>
      <Navbar />
      <Routes>
        <Route
          path="/"
          element={<Home />}
        />
        <Route
          path="/admin"

```

```
        element={user ? <Administracion /> : <Navigate to="/login" />}  
      />  
    <Route  
      path="/login"  
      element={<Login />}  
    />  
  </Routes>  
</div>  
)  
};  
export default App;
```

La protección de la ruta **/admin** consiste en validar el valor de la constante `user` para decidir renderizar el componente Administración o de lo contrario, usar el componente `Navigate` de React Router DOM para redireccionar el usuario a **/login**.

```
<Route  
  path="/admin"  
  element={user ? <Administracion /> : <Navigate to="/login" />}  
/>
```

La protección de rutas consiste finalmente en la condición de una variable o expresión que quisiéramos usar para restringir el acceso a una ruta

Ahora, probemos nuestra aplicación en el navegador y veremos que al intentar acceder a la ruta del componente **Administración**, seremos redireccionados inmediatamente a la ruta del componente **Login**



[Home](#) [Administracion](#) **[Login](#)**

Ingresa a tu cuenta....

Imagen 6. Demostración de la protección de la ruta Administración
Fuente: Desafío Latam

Este redireccionamiento es efectivo incluso ingresando desde la URL a la ruta protegida.

En este ejemplo, para poder acceder a la Administración, entonces basta con asignarle un valor a la propiedad **user** del contexto.

```
import { createContext, useState } from "react";

export const UserContext = createContext();

const UserProvider = ({ children }) => {

  const [user, setUser] = useState({
    email: "desafiolatam@desafiolatam.com",
    displayName: "Desafío Latam",
  });

  return (
    <UserContext.Provider value={{ user, setUser }}>
      {children}
    </UserContext.Provider>
  );
};

export default UserProvider;
```



[Home](#) [Administración](#) [Login](#)

Bienvenido a la administración

Imagen 7. Demostración del acceso a la ruta Administración
Fuente: Desafío Latam



¡Manos a la obra! - Identifíquese

Continúa el ejercicio anterior sobre rutas protegidas, agregando la posibilidad de registrar usuarios y poder iniciar sesión.

Usa tu creatividad para agregar las funcionalidades que creas correspondientes.

Preguntas de cierre

Reflexiona:

- ¿Qué diferencia tienen los componentes **Link** y **NavLink**?
- ¿Qué utilidades nos provee el redireccionamiento programático?
- ¿Cómo se definen los parámetros al momento de declarar un **path** en una ruta?
- ¿Qué nos devuelve la ejecución del hook **useParams()**?
- ¿Cuál es la lógica detrás de la protección de rutas?

